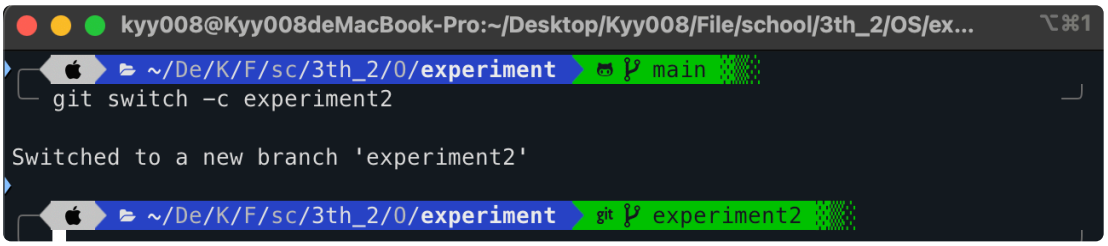
2026 春《操作系统》课程实验报告

实验 2

23301037 客昱阳

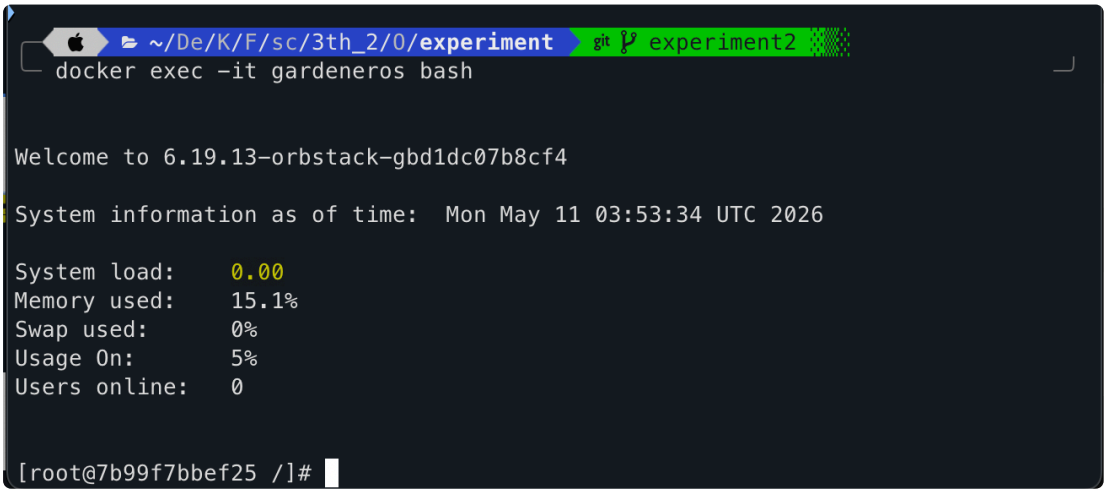
一.准备工作

- 将分支切换到实验一所需的 `experiment2` 分支



- 检查一下上次环境配置的 docker 容器已经启动

- 进入容器

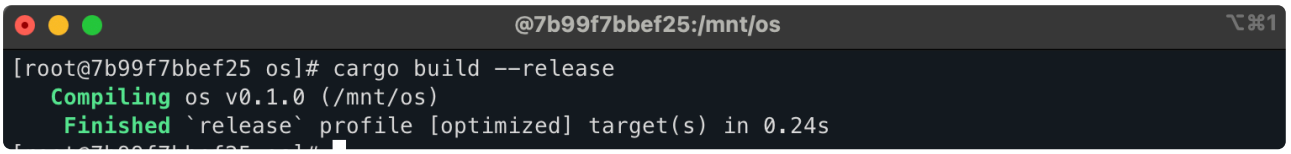


二.实验步骤

1.编译生成内核镜像

- 编译当前实验 1 留下的内核程序，生成 release 版本的 ELF 文件，作为后续转换为裸机镜像的输入

```Bash
cargo build --release
```



- 把刚刚编译生成的 ELF 文件转换成裸机可直接加载的二进制镜像 `os.bin`

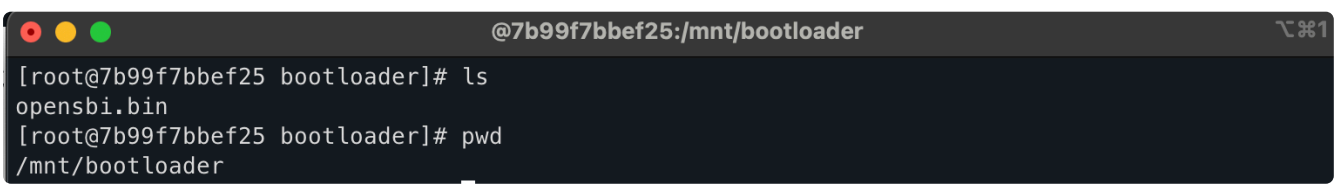
```Bash
rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/os --strip-all -O binary target/riscv64gc-unknown-none-elf/release/os.bin
```

- 准备 OpenSBI 启动文件。QEMU 后面启动裸机内核时，需要先加载 `opensbi.bin` 作为 BIOS。

先在 `os` 的同级目录创建 `bootloader` 目录：

```Bash
mkdir -p ../bootloader
```

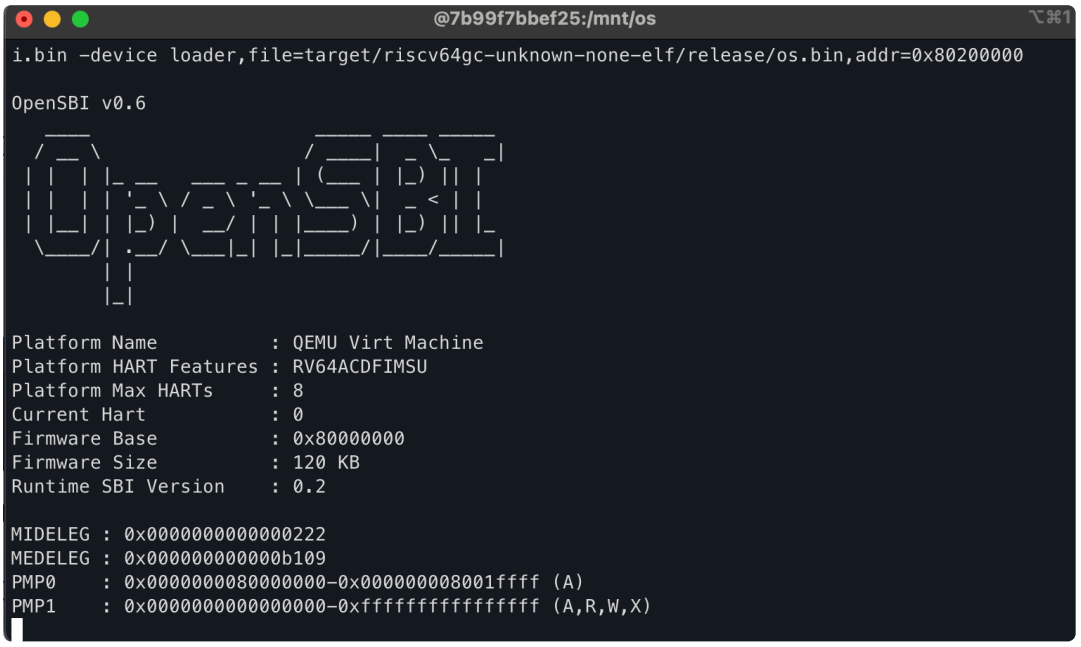
把老师提供的 `opensbi.bin` 放到这个目录下



- 使用 OpenSBI 和刚生成的 `os.bin` 启动 QEMU，测试当前裸机镜像能否被加载运行。

Bash

```
qemu-system-riscv64 -machine virt -nographic -bios ../bootloader/opensbi.bin -device loader,file=target/riscv64gc-unknown-none-elf/release/os.bin,addr=0x80200000
```



可以看到会进入死循环，因为当前 ELF 入口地址还没有改到约定的 `0x80200000`，所以看到 OpenSBI 输出后卡住是正常的。

2.指定内存布局

- 修改 Cargo 配置，让编译器使用我们后面要写的链接脚本 `src/linker.ld`，从而指定内核的内存布局。

Bash

```
vim .cargo/config.toml
```

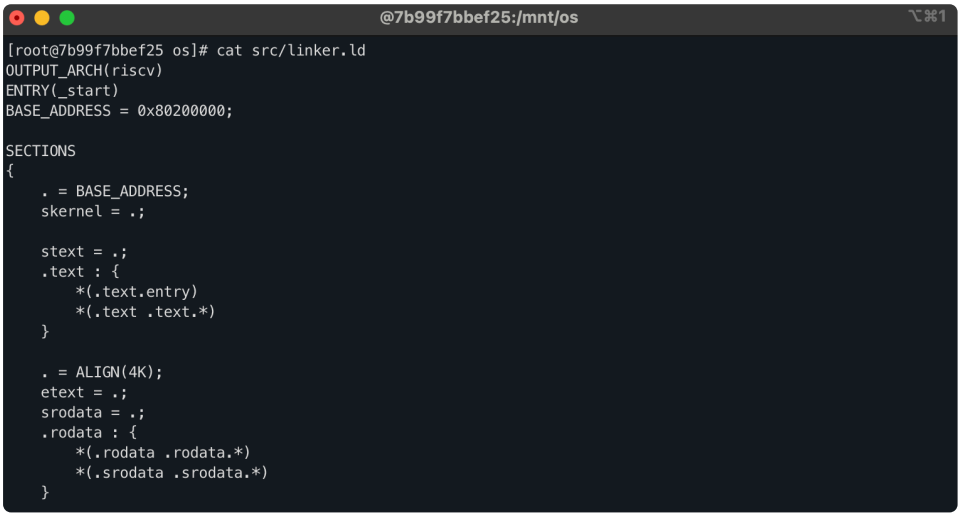
修改为如下

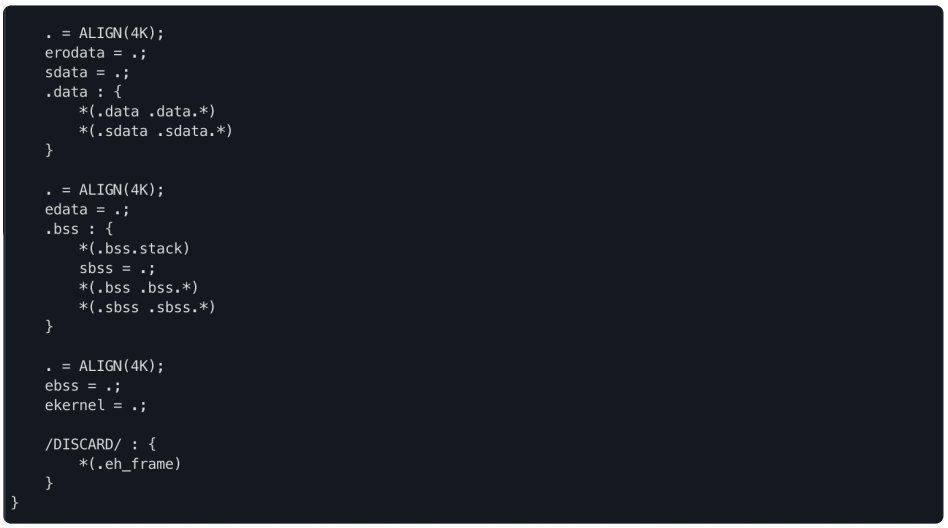
- 创建链接脚本 `src/linker.ld`，把内核入口和各段地址布局固定到 `0x80200000` 开始的位置。

Bash

```
1 vim src/linker.ld
```

写入如下内容





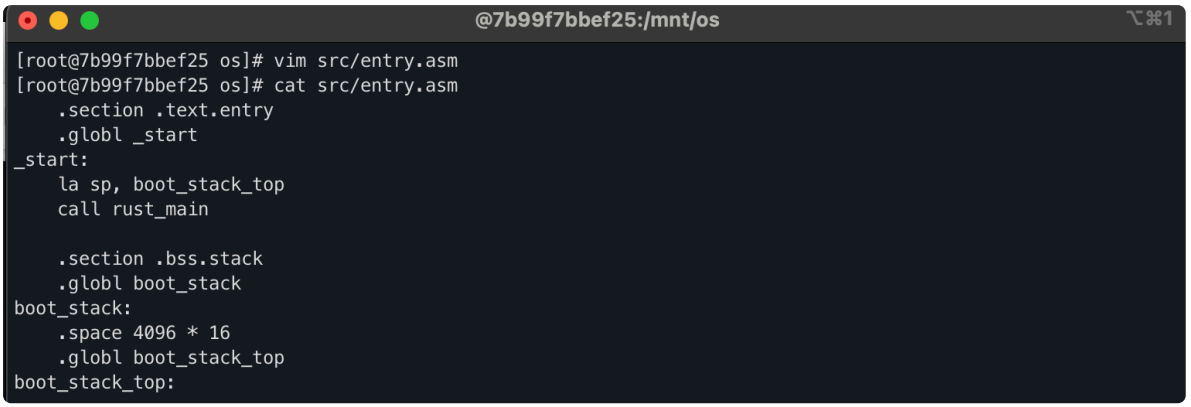
3.配置栈空间布局

- 创建入口汇编文件 `src/entry.asm`，设置启动栈，并从汇编入口 `_start` 跳转到 Rust 入口函数 `rust_main`

Bash

```
1 vim src/entry.asm
```

写入如下内容



- 修改 `src/main.rs`，嵌入刚刚写好的 entry.asm，并把 Rust 入口函数改成 `rust_main`

```bash
1 > src/main.rs
2 vim src/main.rs
```

注意，这里要采用新版 Rust 的安全属性写法，与教程不同

```bash
1 #[unsafe(no_mangle)]
2 pub fn rust_main() -> ! {
```

写入如下内容

```
[root@7b99f7bbef25 os]# cat src/main.rs
#![no_std]
#![no_main]

use core::arch::global_asm;
use core::panic::PanicInfo;

global_asm!(include_str!("entry.asm"));

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}

#[unsafe(no_mangle)]
pub fn rust_main() -> ! {
    loop {}
}
```

- 重新查看 ELF 文件头，确认入口地址已经被链接脚本改成 `0x80200000`。

```bash
rust-readobj -h target/riscv64gc-unknown-none-elf/release/os
```

```
Machine: EM_RISCV (0xF3)
Version: 1
Entry: 0x80200000
ProgramHeaderOffset: 0x40
SectionHeaderOffset: 0x2428
```

表明内存布局设置成功

4.清空 bss 段

- 给内核加入清空 `.bss` 段的函数，保证未初始化的全局数据在内核运行前被置零。

```bash
vim src/main.rs
```

在 `panic` 函数前写入如下内容

注意，由于新版 Rust 的语法要求，这里 `extern "C"` 块也必须标记为 `unsafe`

```
fn clear_bss() {
    unsafe extern "C" {
        fn sbss();
        fn ebss();
    }
    (sbss as usize..ebss as usize).for_each(|a| unsafe {
        (a as *mut u8).write_volatile(0)
    });
}
```

把 `rust_main` 改成如下内容

```
#[unsafe(no_mangle)]
pub fn rust_main() -> ! {
    clear_bss();
    loop {}
}
```

5.实现裸机打印输出信息

- 创建 `sbi.rs`，把裸机环境下需要的 OpenSBI 调用封装起来，用来实现字符输出和关机。

```bash
vim src/sbi.rs
```

写入如下内容

```
[root@7b99f7bbef25 os]# cat src/sbi.rs
#![allow(unused)]

use core::arch::asm;

const SBI_SET_TIMER: usize = 0;
const SBI_CONSOLE_PUTCHAR: usize = 1;
const SBI_CONSOLE_GETCHAR: usize = 2;
const SBI_CLEAR_IPI: usize = 3;
const SBI_SEND_IPI: usize = 4;
const SBI_REMOTE_FENCE_I: usize = 5;
const SBI_REMOTE_SFENCE_VMA: usize = 6;
const SBI_REMOTE_SFENCE_VMA_ASID: usize = 7;
const SBI_SHUTDOWN: usize = 8;

#[inline(always)]
fn sbi_call(which: usize, arg0: usize, arg1: usize, arg2: usize) -> usize {
    let mut ret;
    unsafe {
        asm!(
            "ecall",
            in("x10") arg0,
            in("x11") arg1,
            in("x12") arg2,
            in("x17") which,
            lateout("x10") ret
        );
    }
    ret
}
```

```
pub fn console_putchar(c: usize) {
    sbi_call(SBI_CONSOLE_PUTCHAR, c, 0, 0);
}

pub fn console_getchar() -> usize {
    sbi_call(SBI_CONSOLE_GETCHAR, 0, 0, 0)
}

pub fn shutdown() -> ! {
    sbi_call(SBI_SHUTDOWN, 0, 0, 0);
    panic!("It should shutdown!");
}
```

- 创建 `console.rs`，基于刚才封装的 `console_putchar` 实现裸机环境下的 `print!` 和 `println!` 宏。

Bash

```
1 vim src/console.rs
```

写入如下内容

@7b99f7bbef25:/mnt/os ⌥⌘1

```
[root@7b99f7bbef25 os]# vim src/console.rs
[root@7b99f7bbef25 os]# cat src/console.rs
use crate::sbi::console_putchar;
use core::fmt::{self, Write};

struct Stdout;

impl Write for Stdout {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        for c in s.chars() {
            console_putchar(c as usize);
        }
        Ok(())
    }
}

pub fn print(args: fmt::Arguments) {
    Stdout.write_fmt(args).unwrap();
}

#[macro_export]
macro_rules! print {
    ($fmt: literal $(, $($arg: tt)+)?) => {
        $crate::console::print(format_args!($fmt $(, $($arg)+)?));
    }
}

#[macro_export]
macro_rules! println {
    ($fmt: literal $(, $($arg: tt)+)?) => {
        $crate::console::print(format_args!(concat!($fmt, "\n") $(, $($arg)+)?));
    }
}
```

- 修改 `main.rs`，引入 `sbi` 和 `console` 两个模块，并在 `rust_main` 中输出一条测试信息。

Bash

```
vim src/main.rs
```

```
#![no_std]
#![no_main]

#[macro_use]
mod console;
mod sbi;

use core::arch::global_asm;
use core::panic::PanicInfo;
```

```
#[unsafe(no_mangle)]
pub fn rust_main() -> ! {
    clear_bss();
    println!("Hello, world!");
    loop {}
}
```

重新编译并生成最新的 `os.bin`，把加入 SBI 打印后的内核转换成 QEMU 可加载的裸机镜像。

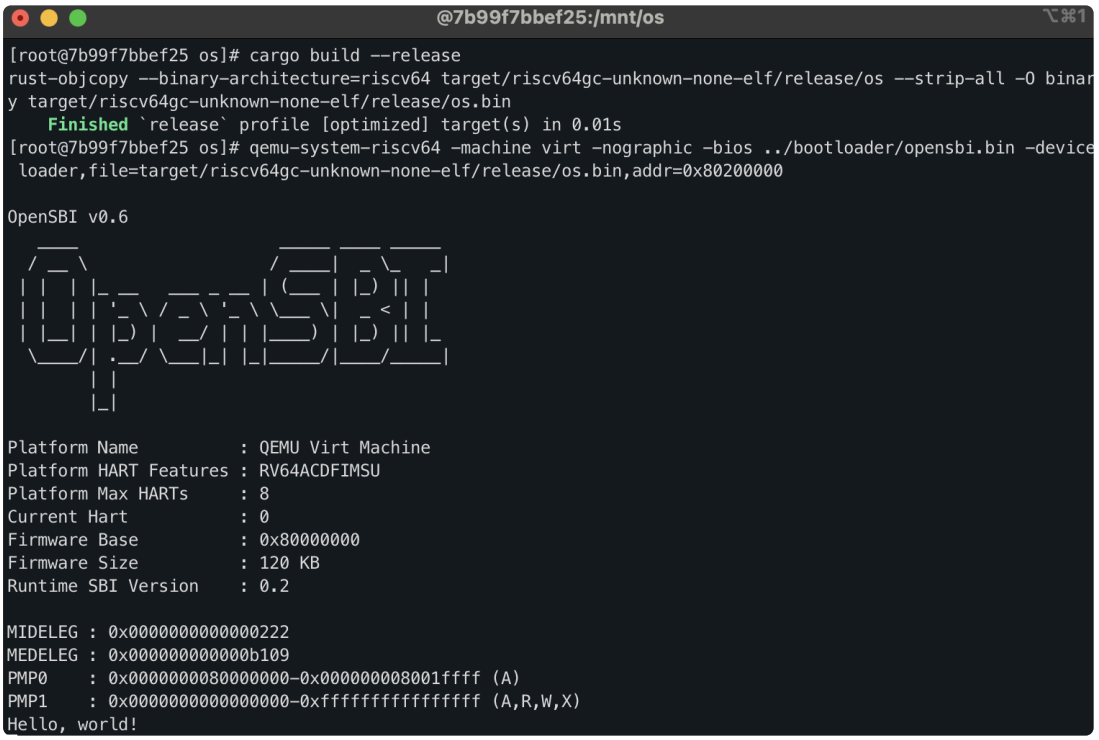
Bash

```
cargo build --release
rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/os --strip-all -O binary target/riscv64gc-unknown-none-elf/release/os.bin
```

运行 QEMU，验证裸机环境下的 SBI 打印可以输出 Hello, world!。

Bash

```
qemu-system-riscv64 -machine virt -nographic -bios ../bootloader/opensbi.bin -device loader,file=target/riscv64gc-unknown-none-elf/release/os.bin,addr=0x80200000
```



6.给异常处理增加输出信息

最后，再给异常处理函数 panic 增加输出显示。

- 创建 `lang_items.rs`，实现 `panic_handler`，让内核 panic 时能打印错误位置和信息，并调用 SBI 关机。

Bash

```
vim src/lang_items.rs
```

写入如下内容

```
[root@7b99f7bbef25 os]# cat src/lang_items.rs
use crate::sbi::shutdown;
use core::panic::PanicInfo;

#[panic_handler]
fn panic(info: &PanicInfo) -> ! {
    let msg = info.message().as_str().unwrap_or("(no message)");

    if let Some(location) = info.location() {
        println!(
            "Panicked at {}:{} {}",
            location.file(),
            location.line(),
            msg
        );
    } else {
        println!("Panicked: {}", msg);
    }

    shutdown()
}
```

7. 修改main.rs输出测试信息

- 使用刚创建的 `lang_items` 模块里的 `panic` 处理函数，并删除 `main.rs` 里原来的 `panic_handler`。

Bash

```
1 vim src/main.rs
```

将文件修改为如下内容

```
[root@7b99f7bbef25 os]# > src/main.rs
vim src/main.rs
[root@7b99f7bbef25 os]# cat src/main.rs
#![no_std]
#![no_main]

#[macro_use]
mod console;
mod lang_items;
mod sbi;

use core::arch::global_asm;

global_asm!(include_str!("entry.asm"));

fn clear_bss() {
    unsafe extern "C" { fn sbss(); fn ebss(); }
    (sbss as *const () as usize..ebss as *const () as usize)
        .for_each(|a| unsafe { (a as *mut u8).write_volatile(0) });
}

#[unsafe(no_mangle)]
pub fn rust_main() -> ! {
    unsafe extern "C" {
        fn stext(); fn etext(); fn srodata(); fn erodata();
        fn sdata(); fn edata(); fn sbss(); fn ebss();
        fn boot_stack(); fn boot_stack_top();
    }
    clear_bss();
    println!("Hello, world!");
    println!(".text [{:#x}, {:#x})", stext as *const () as usize, etext as *const () as usize);
    println!(".rodata [{:#x}, {:#x})", srodata as *const () as usize, erodata as *const () as usize);
    println!(".data [{:#x}, {:#x})", sdata as *const () as usize, edata as *const () as usize);
    println!("boot_stack [{:#x}, {:#x})", boot_stack as *const () as usize, boot_stack_top as *const () as usize);
    println!(".bss [{:#x}, {:#x})", sbss as *const () as usize, ebss as *const () as usize);
    println!("Hello, world!");
    panic!("Shutdown machine!");
}
```

- 重新编译并生成最新的裸机二进制镜像

Bash

```
cargo build --release
rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/os --strip-all -O binary target/riscv64gc-unknown-none-elf/release/os.bin
```

- 运行 QEMU，验证最终效果：打印各段地址，然后触发 `panic` 输出错误信息，并通过 SBI 关机退出。

Bash

```
qemu-system-riscv64 -machine virt -nographic -bios ../bootloader/opensbi.bin -device loader,file=target/riscv64gc-unknown-none-elf/release/os.bin,addr=0x80200000
```

```
[root@7b99f7bbef25 os]# qemu-system-riscv64 -machine virt -nographic -bios ../bootloader/opensbi.bin -device loader,file=target/riscv64gc-unknown-none-elf/release/os.bi
n,addr=0x80200000

OpenSBI v0.6
   ____                  _____ ____ _____
  / __ \                / ____|  _ \_   _|
 | |  | |_ __   ___ _ __ | (___ | |_) || |
 | |  | | '_ \ / _ \ '_ \ \___ \|  _ < | |
 | |__| | |_) |  __/ | | |____) | |_) || |_
  \____/| .__/ \___|_| |_|_____/|____/_____|
        | |
        |_|

Platform Name          : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs     : 8
Current Hart           : 0
Firmware Base          : 0x80000000
Firmware Size          : 120 KB
Runtime SBI Version    : 0.2

MIDELEG : 0x0000000000000222
MEDELEG : 0x000000000000b109
PMP0    : 0x0000000080000000-0x000000008001ffff (A)
PMP1    : 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
Hello, world!
.text [0x80200000, 0x80201000)
.rodata [0x80201000, 0x80202000)
.data [0x80202000, 0x80202000)
boot_stack [0x80202000, 0x80212000)
.bss [0x80212000, 0x80212000)
Hello, world!
Panicked at src/main.rs:34 Shutdown machine!
```

- 为了之后方便编译运行，编写 `Makefile`，把编译、生成 `os.bin`、运行 QEMU 这些命令封装起来，后面可以直接用 `make run` 运行内核。

Bash

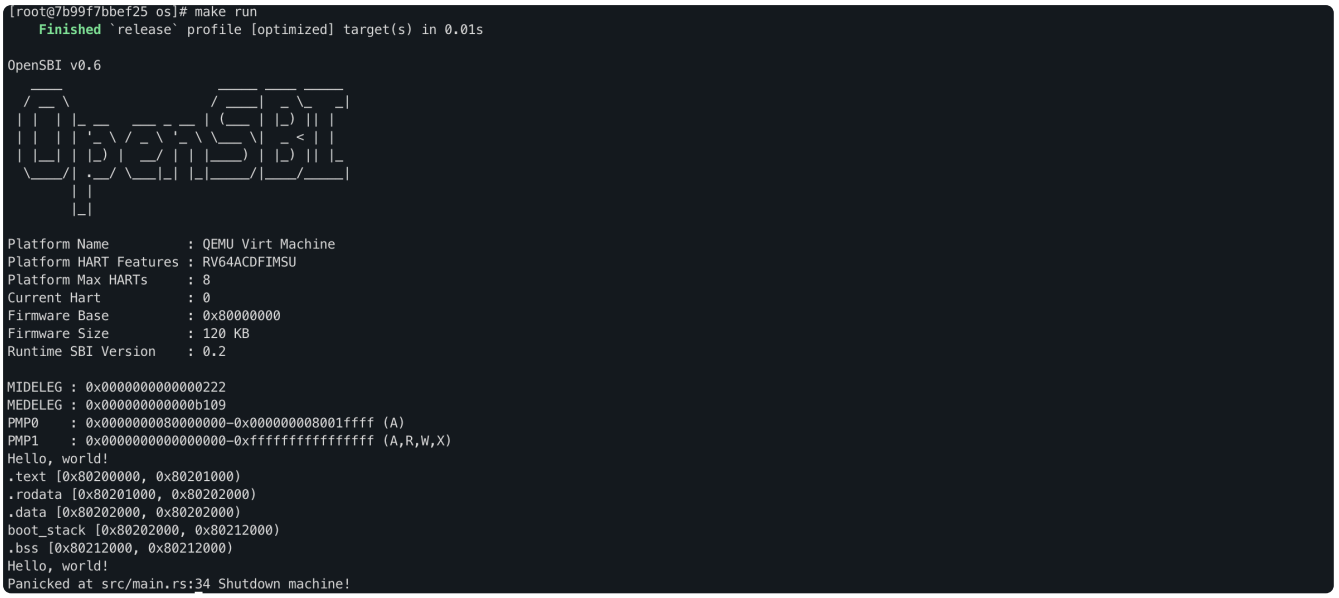
```
vim Makefile
```

写入如下内容

```
[root@7b99f7bbef25 os]# cat Makefile
TARGET := riscv64gc-unknown-none-elf
MODE := release
KERNEL_ELF := target/$(TARGET)/$(MODE)/os
KERNEL_BIN := $(KERNEL_ELF).bin
DISASM_TMP := target/$(TARGET)/$(MODE)/asm

SBI ?= opensbi
BOOTLOADER := ../bootloader/$(SBI).bin

KERNEL_ENTRY_PA := 0x80200000

OBJDUMP := rust-objdump --arch-name=riscv64
OBJCOPY := rust-objcopy --binary-architecture=riscv64

DISASM ?= -x

build: $(KERNEL_BIN)

env:
        (rustup target list | grep "riscv64gc-unknown-none-elf (installed)") || rustup target add $(TARGET)
        cargo install cargo-binutils
        rustup component add rust-src
        rustup component add llvm-tools-preview

$(KERNEL_BIN): kernel
        @$(OBJCOPY) $(KERNEL_ELF) --strip-all -O binary $@
```

```
kernel:
        @cargo build --release

clean:
        @cargo clean

disasm: kernel
        @$(OBJDUMP) $(DISASM) $(KERNEL_ELF) | less

disasm-vim: kernel
        @$(OBJDUMP) $(DISASM) $(KERNEL_ELF) > $(DISASM_TMP)
        @vim $(DISASM_TMP)
        @rm $(DISASM_TMP)

run: build
        @qemu-system-riscv64 \
                -machine virt \
                -nographic \
                -bios $(BOOTLOADER) \
                -device loader,file=$(KERNEL_BIN),addr=$(KERNEL_ENTRY_PA)

.PHONY: build env kernel clean disasm disasm-vim run
```

验证使用 `make run` 命令可以正常运行



三.问题回答

1.分析 linker.ld 和 entry.asm 所完成的功能；

(1) `inker.ld` 的核心作用是规定内核从 `0x80200000` 开始运行，并安排各个段在内存中的位置。

Bash

```
1 OUTPUT_ARCH(riscv)
2 ENTRY(_start)
3 BASE_ADDRESS = 0x80200000;
```

- 指定目标架构是 RISC-V。
- 指定程序入口是 _start。
- 指定内核起始地址为 0x80200000。

Bash

```
1 .text : {
2     *(.text.entry)
3     *(.text .text.*)
4 }
```

- 把 `.text.entry` 放在代码段最前面，使入口代码最先执行。

```
Bash

1 .rodata
2 .data
3 .bss
```

(2) `entry.asm` 的核心作用是建立程序最早执行的入口代码，并设置启动栈。

```
Bash

1 .section .text.entry
2 .globl _start
3 _start:
4     la sp, boot_stack_top
5     call rust_main
```

- 定义 `_start` 入口，与 `linker.ld` 中的 `ENTRY(_start)` 对应。
- 设置栈指针 `sp`。
- 跳转到 Rust 代码中的 `rust_main`。

```
Bash

1 .section .bss.stack
2 boot_stack:
3     .space 4096 * 16
4 boot_stack_top:
```

- 在 `.bss` 段中分配 64KB 栈空间。
- `boot_stack_top` 作为栈顶地址供启动代码使用。

2.分析 sbi 模块和 lang_items 模块所完成的功能；

(1) `sbi.rs` 的主要作用是让内核在裸机环境下可以调用 OpenSBI 提供的功能

```Bash
fn sbi_call(...) {
    asm!("ecall", ...);
}
```

- 通过 `ecall` 进入 OpenSBI。
- 为后面的输出和关机功能提供基础。

```Bash
pub fn console_putchar(c: usize)
pub fn shutdown() -> !
```

- `console_putchar` 用来向控制台输出字符。
- `shutdown` 用来关闭 QEMU 虚拟机。

(2) `lang_items.rs` 的主要作用是在 `no_std` 环境下实现 `panic` 处理函数。

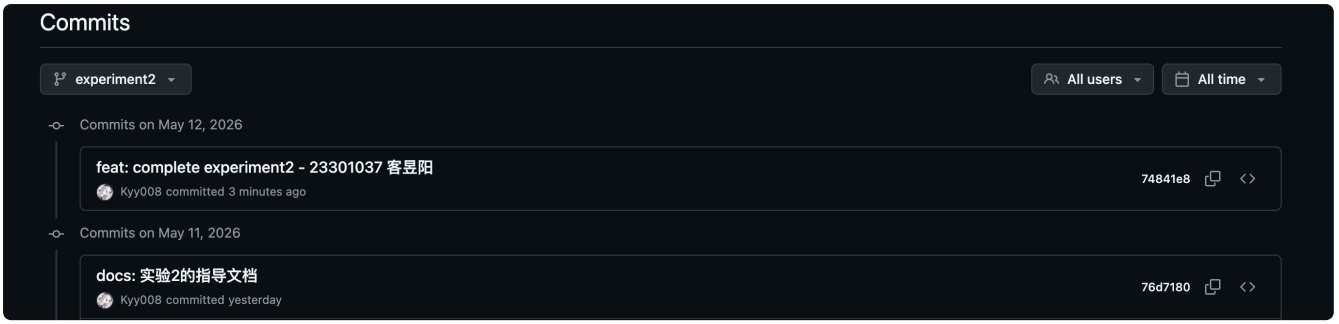
```Bash
#[panic_handler]
fn panic(info: &PanicInfo) -> ! {
```

- 当内核发生 `panic!` 时，程序会进入这个函数。

```Bash
println!("Panicked at ...");
shutdown()
```

- 输出 `panic` 的位置和信息。
- 输出完成后调用 `shutdown` 关机。

四.Git 提交截图


Commits
experiment2
All users
All time
Commits on May 12, 2026
feat: complete experiment2 - 23301037 客昱阳
Kyy008 committed 3 minutes ago
74841e8
Commits on May 11, 2026
docs: 实验2的指导文档
Kyy008 committed yesterday
76d7180

五.其他说明

无